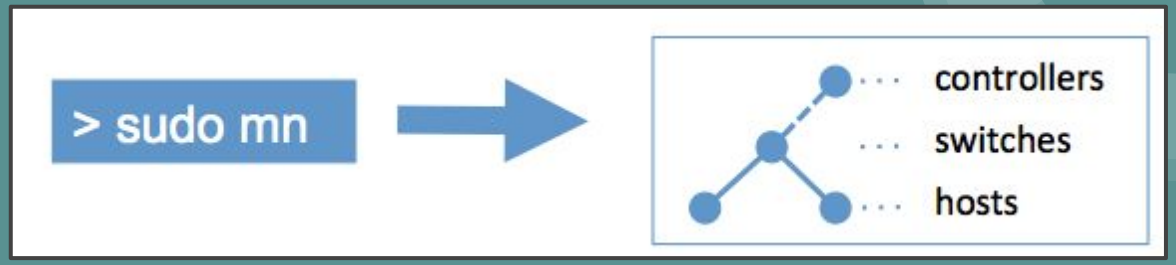
Introduction to Mininet



Jai Singh

Mininet

Introduction to Mininet

1. Mininet is an emulator for rapid prototyping of Software Defined Networks.
2. It is basically a network emulator that allows you to emulate an entire computer network (Hosts, Links, Switches) on a single machine.
3. It is primarily used as a learning tool to test, experiment, and learn about software-defined networks. Mininet is preferable because it is very fast (can run on a laptop) and helps us to create customizable topologies.

How does it work?

1. Mininet is similar to how containerization technologies like docker, podman work.
2. Just like docker, mininet relies on process-based virtualization (kernel same, root different with isolation) and network namespaces to create a virtual network.
3. In mininet, hosts (basically your vms) are emulated as bash processes running in a network namespace. So any application that runs on your host machine can easily run inside Mininet hosts as well.
4. The only difference is that Mininet hosts can only see the process they spawned and have a private network interface to work with.

Common Terms related to Mininet

1. **Hosts :** Think of hosts like VM with their own private network interface. You can run any type of applications inside the Hosts. Hosts correspond to real life machines connected to the network.
2. **Links :** Links can be thought of as cables that connect between hosts and switches. Internally, they reside in the linux kernel and connect our emulated switch to emulated host.
3. **Switch :** Just like real world switches, switches in Mininet are emulated switches that are used for forwarding data from one device to another device.

Features of Mininet

1. Command line launcher (mn) to instantiate the network
2. Python API for creating networks of varying size and topologies
3. CLI for interacting with and diagnosing the network.

Setting up Mininet

Setting Up Mininet

Mininet can be setup in two ways

- Installation via package manager
- Source Based Install
- Mininet VM Images

Installation via Package Manager

1. If you are running a recent version of ubuntu/debian run,
2. **sudo apt install mininet** to install mininet
3. **sudo apt-get install openvswitch-switch** (Install Open V Switch)
4. **sudo service openvswitch-switch start** (Running the service, use enable to start everytime)
5. To Install Open Flow reference switch, wireshark dissector and reference controller
6. **git clone https://github.com/mininet/mininet**
7. **mininet/util/install.sh -fw**

Installation from source

1. To install natively from source :
2. **git clone https://github.com/mininet/mininet**
3. **cd mininet**
4. **git tag # list available versions**
5. **git checkout -b mininet-2.3.0 2.3.0**
6. **cd ..**
7. **Run mininet/util/install.sh [options]**
8. [options] : -a (All the components) -h(help)

To install Mininet + user switch + OvS (using your home dir):

- Step 7 with **-nfv** as the option

Mininet VM (Preferred)

- Mininet also provides Virtual Appliance / VM Images (OVA file) with all the necessary tools installed that can be directly imported and run by your Hypervisor.
- Hypervisors that work with Mininet VM
  - Vmware
  - Virtualbox
  - Hyper V
  - Qemu

Steps to setup a Mininet VM

- Download the Mininet VM image from https://github.com/mininet/mininet/releases (ovf.zip file) for your hypervisor.
- Unzip the file to get the `ova` and `vmdk` image file
- If you are using virtual box, double click the ova file and complete the setup keeping the defaults to create your VM. (Virtualbox will automatically create a vm with the disk imported
- If you are using vmware, open vmware workstation player and click on **File->Open** to open the File Dialog. Now double click on the ova file inside the file dialog to import the VM.
- Vmware just like virtual box will automatically import the disks.
- The user name and password for the VM is `mininet` and `mininet`

Additional Notes

- For installing in other hypervisors, configuring ssh, please go through

http://mininet.org/vm-setup-notes/

Creating a Simple Network in Mininet

Creating a Simple Network in Mininet

1. **sudo mn**

This command is used to create a network in mininet with two hosts and a switch automatically connected to a sdn controller which is inbuilt in the application.

2. **nodes**

This Command is used to gives us the nodes of the network.

3. **links**

This command is used to give us the links in the network.

4. **ports**

This command is used to view the ports in the network.

Creating a Simple Network in Mininet

5. **intfs**

This command is used to know the interfaces in the network.

6. **dump**

This command is used to list the information about the nodes,interfaces,ip addresses of the hosts,ports etc.

7. **pingall**

This command is used us to know whether a single host can reach all other hosts in the network.

Creating a Simple Network in Mininet

8. **pingallfull**

This command is used to give the reachability of hosts in details , as after running the command we can see also get the RTT in details.

9. **[host1] ping [host2]**

This command is used to manually test the two hosts.it will run in infinite loop.

10. **[host1] ping [host2] -c num**

This command is used to manually test the two host and it will run num times.

Creating a Simple Network in Mininet

11. **sudo mn -c**

This Command is used to ensure that it has been shut down properly or to clean up after an expected crash.

12. **help**

This command is used to know about the documented command of mininet.

Data Transfer Between Hosts

1. Data Transfer between hosts is done using a simple python server and a client application or Iperf command which is a proper network performance measurement tool which is being used prominently by the network performance analyst.

2. For the Data Transfer , one an either enter command on the mininet CLI itself or we can open the separate terminals for the hosts and enter the commands.

3. If we are running our command in CLI then we have to mention the host name and the respective command of the client or server.

Example 1 (Using Iperf inside mininet)

1. In our Example , we are going to use Iperf it is a tool which is used to measure maximum TCP and UDP bandwidth performance. We are using it for the packet transfer (here).

(i) start xterm for both hosts **h1** and **h2** , using the command : **xterm h1 h2**

(ii) Now in **h1** run **iperf -s -u -i 1** to start a UDP server which sends packet at an interval of 1 second.

(iii) now then in **h2** run **iperf -c 10.0.0.1 -u -b 1m -n 1000** which creates a UDP client that connect to **h1** at address 10.0.0.1 bandwidth=1M and number of bytes to transfer 1000.

Example 2 (Client - Server)

In this example we are going to transfer a file . host **h1** will acts as a client and **h2** will acts as a server,

(i) start xterm for both hosts **h1** and **h2 ,** using the command : **xterm h1 h2**

(ii) now in **h1** run **python -m http.server 80 &.**

(ii) now in **h2** run **wget -o - 10.0.0.1**.

And your Index.html got transferred :) .

dpctl command

(i) dpctl is a program,command line tool for monitoring OpenFlow Switch.it can show the flows,features,configuration,table entries, etc. of the switches.

(ii) it can work with any openFlow switches and basically a switch management utility.

(ii) to see how it works , let's make a simple network with two hosts and two switches.

run the command : **sudo mn –mac – topo linear,2**

(iv) **dpctl show :** this command will show you the information regarding the switch id i.e dpid , the number of tables , buffers in the switches

dpctl command

(v) **dpctl dump-desc :** this command will give the switch information.Here the Mininet uses the Open vSwitch.

(vi) **dpctl dump-ports :** this command will give you the ports information the local ports, interfaces, (main) ports which are being used.

(vii) **dpctl dump-ports-desc :** this command will give you the detailed statistics of the network devices attached to the switch.

(viii) **dpctl dump-flows :** this command will give you the flow entries of the switch.Initially there is no flows as there is no communication between the hosts.

dpctl command

(ix) **dpctl dump-tables :** whenever a packet is received by the switch it tries to match the entries like what is in the in_port , what is the source and destination of the packet so if this is matched in the switch it can say i can forward it to a nearby switch so it can reach it's destination.Initially there are no packets flow because there is no packet flow between the hosts.

Now let's transfer the packet between hosts **h1** and **h2** and here we are doing TCP transfer.

(i) first run the command : **xterm h1 h2** on the mininet CLI.

(ii) on **h1** host type the command : **iperf -s**

(iii) on **h2** type the command : **iperf -c 10.0.0.1 -t -3 -i 1**

dpctl command

(x) **dpctl dump-aggregate :** This command is used to find the flow count(on the switch) at a particular time.

Topologies in Mininet

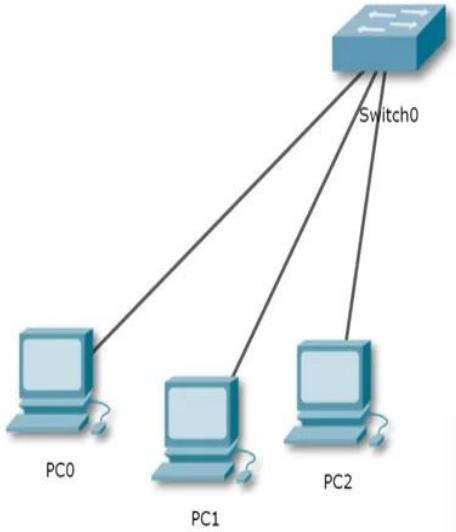
(i) the default topology in mininet consist of two hosts and two switches. There are other topologies which are available in mininet which can be accessed using the topo command.

(ii) some of the most common topologies are as follows : **Single, Linear , Reversed,Tree , Torus.**

Single Topology

single : this keyword will create a topology with a single switch attached to the number of hosts that are mentioned by the users.

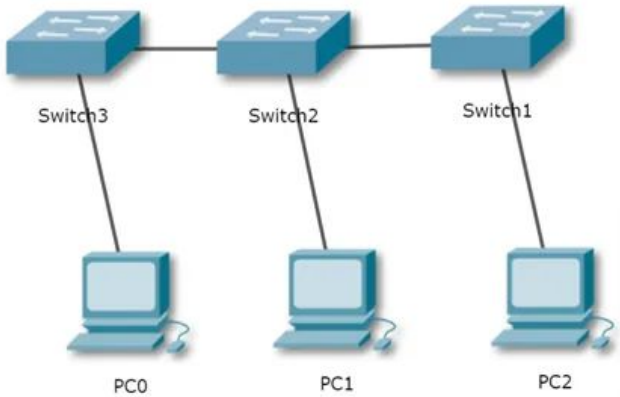
Command : **sudo mn –topo single,[number of hosts]**



Linear Topology

linear : this creates a topology in linear fashion with the specifies number of switches and number of hosts for each switch.

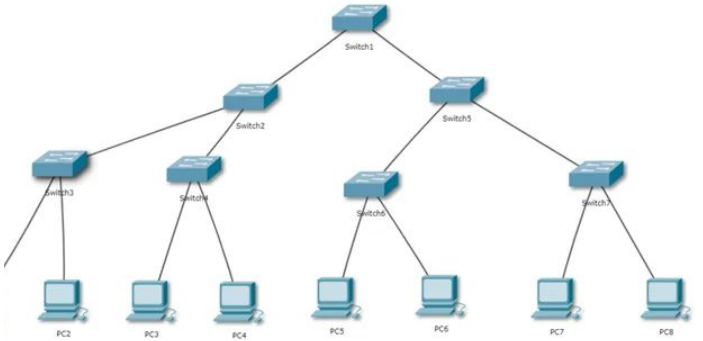
Command : **sudo mn –topo linear,[number of switches] , [number of hosts for each switches]**



Tree Topology

tree : this keyword creates a tree topology of specifies depth of switches.

Command : **sudo mn –topo tree,[depth],[number of children]**



Reversed Topology

reversed: this creates a topology similar to the single one but the interfaces or links are assigned in a reverse fashion.

Command : **sudo mn –topo reversed,[number of host]**

Torus Topology

torus : this keyword will create a torus topology with specified length and breadth.

Command : **sudo mn –topo torus,[length],[breadth]**

Custom Topology

(1) For Creating a Custom Topology Involves writing a python script that defines the hosts, switches and links in your network.

(2) Some Common Methods in Topo :

(a) addHost('host') : this method is used to add host to our network topology.

(b) addLink(node1,node2) : this method is used to add link between two nodes(switch or hosts) in our network topology.

(c) addSwitch('switch') : this method is used to add switch in out network topology.

Custom Topology

(3) Please find below the Link of the Python Script :

Python Script : Mininet Python Script

(4) Command to Execute the Topology :

sudo mn --custom /home/js/Mininet_Tuts/Topo.py --topo mytopo

Python Script Example

```
from mininet.topo import Topo

class MyTopo( Topo ):
   "Simple topology example."

   def build( self ):
      "Create custom topo."

      # Add hosts and switches
      leftHost = self.addHost( 'h1' )
      rightHost = self.addHost( 'h2' )
      leftSwitch = self.addSwitch( 's3' )
      rightSwitch = self.addSwitch( 's4' )

      # Add links
      self.addLink( leftHost, leftSwitch )
      self.addLink( leftSwitch, rightSwitch )
      self.addLink( rightSwitch, rightHost )


topos = { 'mytopo': ( lambda: MyTopo() ) }
```

Additional Info

For more information you can go through the content below :

- http://mininet.org/sample-workflow/
- https://github.com/mininet/mininet/wiki/Videos
- http://mininet.org/walkthrough/
- https://github.com/mininet/mininet/wiki/Documentation

References

- Mininet emulator in Software Defined Networks - GeeksforGeeks
- mininet installation on ubuntu
- [Eng sub] Create a simple network in mininet - SDN #2 | mininet tutorial
- http://mininet.org/walkthrough
- http://mininet.org/sample-workflow/